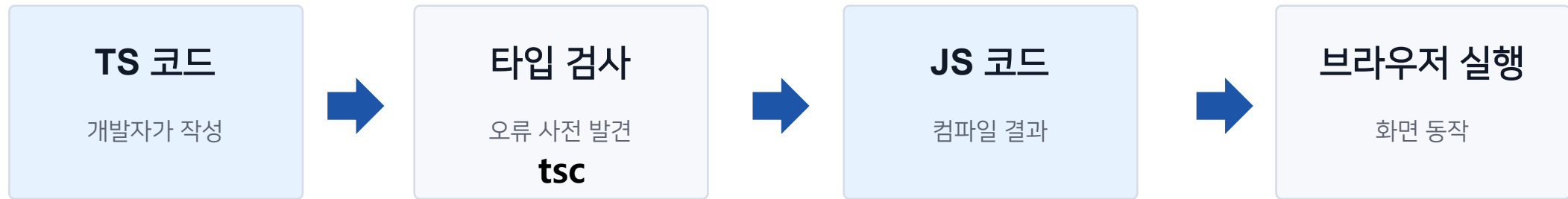


JavaScript와 TypeScript 기본문법 1



JavaScript와 TypeScript의 관계

- TypeScript는 JavaScript에 **타입** 시스템을 추가한 언어이며, 실행 전 JavaScript로 변환
- **타입**은 브라우저에서 직접 실행되는 문법이 아니라 **개발 단계에서 오류를 줄이기 위한 검사 기준**
- 브라우저의 JavaScript 엔진은 TypeScript의 **: number** 같은 타입 문법을 직접 실행하지 않음



예시: 타입만 추가된 TypeScript

```
function add(a: number, b: number): number {  
  return a + b;  
}
```

```
console.log(add(10, 20));
```

변환 후 실행되는 JavaScript

```
function add(a, b) {  
  return a + b;  
}
```

```
console.log(add(10, 20));
```

- 세미 콜론(;)

- 하나의 명령이 끝날 때 세미콜론을 붙여도 되고 붙이지 않아도 됨. **그러나, 붙이기를 권장**
 - 대부분 자바스크립트 엔진이 세미콜론을 자동으로 붙이지만 일부 명령에서는 가끔 에러가 발생
- 한 줄에 여러 명령어를 넣을 때는 세미콜론으로 구분

```
console.log('Hello, World!'); console.log('Hello, JavaScript!'); console.log('Hello, TypeScript!');
```

- 주석(comment)

- 사람만 알아볼 수 있도록 설명을 작성한 부분이며, 코드 실행에 영향은 없음
- 한 줄 주석 : **//** 기호 뒤에 작성

```
console.log('Hello, Comment!')           // Hello, Comment! 출력
```

- 여러 줄 주석 : **/* */** 기호로 감싼, 안쪽에 작성

```
/* console.log('Hello, World!');  
   console.log('Hello, comment!') */
```

- 들여쓰기

- **제한 없음**, 보통 2칸 띄어쓰기
- 규칙적으로 사용하면 코드의 가독성을 높일 수 있음

```
if (condition) {  
    console.log('Hello, world!');  
}
```

Prettier Extension 셋팅 : 47~48 참고

자료형

데이터 타입

JavaScript와 TypeScript의 자료형 개념

- **JavaScript**에서는 변수를 선언할 때 **자료형을 지정하지 않음**
 - 하나의 변수 값(value)에 숫자, 문자열, 불리언 값을 차례로 저장할 수 있음
- **TypeScript**에서는 타입을 명시할 수 있음
 - TypeScript는 코드를 실행하기 전에 타입 오류를 검사

⌞ JavaScript

```
let value = 10;  
value = "hello";  
value = true;
```

⌞ TypeScript

```
let value: number = 10;  
  
value = 20;           // 정상  
value = "hello";     // 오류
```

- 자바스크립트에 존재하는 값
 - 값(value) : 프로그램에서 조작할 수 있는 데이터
 - 자료형(data type) : 값의 종류
- 기초 타입 : String, number, boolean, null, undefined는 화면 출력과 사용자 입력 처리의 기초

JavaScript

```
let userName = "Kim";  
let age = 20;  
let isLogin = true;  
  
let selected = null;  
let result; // undefined
```

TypeScript

```
let userName: string = "Kim";  
let age: number = 20;  
let isLogin: boolean = true;  
  
let selected: string | null = null;  
let result; // undefined
```

기본 타입은 “값의 종류”를 나타내며, TypeScript는 변수·함수·객체에 이 종류를 명확히 붙입니다.

age 는 변수이고 20은 값이다. 값 20의 자료형은 **number** 이다.

undefined

- 변수는 만들었지만 아직 값이 없음
- 함수가 값을 반환하지 않았을 때 자주 등장
- 개발자가 의도적으로 넣기보다 결과로 발생하는 경우가 많음

```
let message;  
console.log(message); // undefined  
console.log(typeof undefined);
```

null

- 개발자가 의도적으로 “현재 값이 없다”고 명시적으로 표현
- 선택된 데이터가 아직 없을 때 사용
- TypeScript에서는 string | null처럼 허용 범위를 표시

```
let selected: string | null = null;  
selected = "React";  
console.log(typeof null);
```

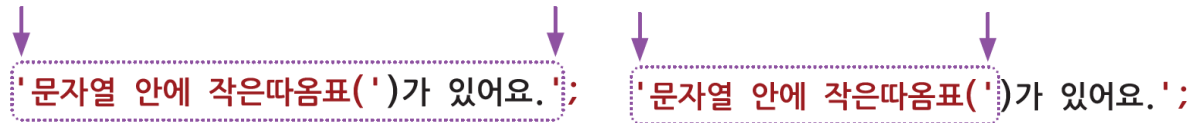
```
> undefined === null;  
< false
```

string(문자열)

- 문자열(string): 문자들이 하나 이상 나열되어 있는 자료형
- 시작과 끝이 따옴표로 감싸진 값
 - 시작과 끝을 같은 종류의 따옴표로 감싸야 함 $\Rightarrow$ "...", '...'

- 문자열 안에 따옴표 사용하기

- 따옴표가 문자열 중간에 들어 있을 때, 문자열을 다른 종류의 따옴표로 감싸야 함


사람이 인식하는 문자열의 시작과 끝 자바스크립트 엔진이 인식하는 문자열의 시작과 끝

- 이스케이핑(escaping): 기호를 다르게 해석하게 하는 행위

- 예: 백슬래시(\)가 붙은 따옴표는 일반 문자로 해석

```
> "문자열 안에 큰따옴표(\"")가 있어요.";
< '문자열 안에 큰따옴표(")가 있어요.'
> '문자열 안에 작은따옴표(\')가 있어요.';
< "문자열 안에 작은따옴표(')가 있어요."
```

string(문자열)

- 한 문자열을 여러 줄로 표시하기
 - 문자열에서 개행 할 부분 앞에 \n 문자 넣기

```
> alert('여러 줄에 걸쳐\n표시됩니다.');
```

chrome://new-tab-page 내용:

여러 줄에 걸쳐
표시됩니다.

확인

- 문자열 합치기
 - 두 문자열 사이에 + 기호를 두면 양쪽 문자열이 하나로 합쳐 짐

```
> '문자열이 긴 경우에는 문자열을 ' + '나눈 뒤 다시 합칩니다.';
```

```
< '문자열이 긴 경우에는 문자열을 나눈 뒤 다시 합칩니다.'
```

string(문자열)

- 템플릿 리터널(Template Literal)
 - 문자열 안에 변수나 계산식을 쉽게 넣기 위한 문법

JavaScript

```
const user = "홍길동";  
const age = 20;  
  
console.log(`이름은 ${user}이고 나이는 ${age}살입니다.`);
```

- 백틱(`, backtick 또는 backquote)으로 감싸진 문자열
 - `` -> 백틱
 - \${ } -> 값 또는 식 삽입
 - > alert(`여러 줄에 걸쳐
표시됩니다.

줄을 더 늘려 볼까요?`);
 - < undefined

chrome://new-tab-page 내용:

여러 줄에 걸쳐
표시됩니다.

줄을 더 늘려 볼까요?

확인

number(숫자)

- 숫자(number): 따옴표 없이 숫자 그대로 작성

```
5;
```

- 따옴표로 감쌀 경우 문자열이 됨

```
> typeof 5;
```

```
< 'number'
```

```
> typeof '5';
```

```
< 'string'
```

- 지수 표기법(exponential notation)을 사용할 수 있음

```
5e4; // 5 * 104(10000) = 50000(+는 생략 가능)
```

```
5e+4; // 5 * 104(10000) = 50000
```

```
5e-3; // 5 * 10-3(1/1000) = 0.005
```

- JavaScript에는 int, float, double 따로 없음. 모두 number임

number(숫자)

- 문자열을 숫자로 바꾸기 (HTML <input>에서 입력 받은 값은 보통 문자열임)
 - parseInt()와 Number() 함수 사용
 - parseInt() 함수: 문자열을 정수로만 바꿈
 - 문자열을 실수로 바꾸고 싶으면 parseFloat() 함수 또는 Number() 함수 사용

JavaScript - parseInt

```
const numString = "100";  
const num = parseInt(numString);  
  
console.log(typeof(numString)); // string  
console.log(numString); // 100  
console.log(typeof(num)); // number  
console.log(num); // 100
```

JavaScript - Number()

```
const a = "10";  
const b = "20";  
  
const result = Number(a) + Number(b);  
console.log("a + b = " + result); // a + b = 30
```

• NaN

- NaN: Not a Number(숫자가 아님)
- 자료형은 number임
- 주의: 이름과는 다르게 숫자

JavaScript - NaN

```
const value = Number("hello");  
  
console.log(value); // NaN  
console.log(typeof value); // number
```

number(숫자)

- 산술 연산자 사용하기
 - +, -, *, / 등의 기호를 사용
 - % 연산자: 나눗셈의 나머지를 구하기
 - ** 연산자: 숫자를 거듭제곱
- 무한 값
 - Infinity: 무한 값
 - - Infinity: 음수 무한 값. 음수를 0으로 나누는 경우
 - NaN: 무한끼리 연산할 때(연산이 성립하지 않는다는 의미)

JavaScript - Infinity

```
console.log(100 / 0); // Infinity  
console.log(-1 / 0); // -Infinity  
console.log(Infinity - Infinity); // NaN
```

- 연산자 우선순위 이해하기
 - 연산자 우선순위가 높을수록 먼저 계산
 - 같은 우선순위의 연산자가 여러 개 나오면 먼저 나온 순서대로 계산

우선순위	연산자(심표로 구분)	우선순위	연산자(심표로 구분)
19	()(그룹화)	9	==, !=, ===, !==
18	., [], new, ()(함수 호출), ?.	8	&
17	new(인수 없이)	7	^
16	++(후위), --(후위)	6	
15	!, ~, +(단항), -(단항), ++(전위), --(전위), typeof, void, delete, await	5	&&
14	**	4	, ??
13	*, /, %	3	? : (삼항 연산자)
12	+(다항), -(다항)	2	=, +=, -=, **=, *=, /=, %=, <<=, >>=, >>>=, &=, ^=, =, &&=, =, ??=, yield, yield*
11	<<, >>, >>>		
10	<, <=, >, >=, in, instanceof	1	, (심표)

boolean(불 값) 및 비교 연산자

- 불 값(boolean): 참(true)과 거짓(false)을 나타내는 자료형

- 불 값 표현하기

- 따옴표로 감싸지 않고 **true**와 **false**를 입력

```
> typeof true;  
< 'boolean'
```

- 비교 연산자 사용하기

- >: 왼쪽 값이 오른쪽 값보다 크다(초과)
 - <: 왼쪽 값이 오른쪽 값보다 작다(미만)
 - >=: 크거나 같다(이상)
 - <=: 작거나 같다(이하)
 - ==: 양쪽 값이 같은지 비교
 - !=: 양쪽 값이 다른지 비교

> 5 > 3;	> 5 >= 5;
< true	> true
> 5 < 3;	> 5 <= 4;
< false	> false

- ==와 ===의 다른 점

- ==: 값이 같은지 비교, 자료형이 달라도 값이 같으면 true, (!=)
 - ===: 값뿐 아니라 자료형까지 같은지 비교, 자료형까지 같을 때만 true, (!==)

특별한 이유가 없다면 ===, != 사용을 기본으로 함

- 논리 연산자 사용하기
 - **&&** 연산자: '그리고'를 의미
 - 왼쪽 식과 오른쪽 식이 모두 true여야 true
 - **||** 연산자: '또는'을 의미
 - 왼쪽 식이나 오른쪽 식 둘 중 하나라도 true면 true
- 논리 연산자 사용 시 유의할 점
 - **&&** 연산자
 - 앞에 나오는 값이 참인 값이면 뒤에 나오는 값 결과, 앞에 나오는 값이 거짓인 값이면 앞에 나오는 값이 결과
 - **||** 연산자
 - 앞에 나오는 값이 참인 값이면 앞에 나오는 값 결과, 앞에 나오는 값이 거짓인 값이면 뒤에 나오는 값이 결과

타입스크립트의 데이터 타입 종류

• 자바스크립트의 데이터 타입

- 자바스크립트: 변수에 모든 데이터 타입 할당 가능, 동적 타이핑(Dynamic Typing)
- 문제: 의도와 다르게 값 변경 가능
- 예시: myname 변수에 문자열 대신 숫자 1 할당 가능

```
let myname = 'lee';  
myname = 1;
```

• 동적 타이핑

- 변수의 타입이 런타임(프로그램 실행 시점)에 결정되고 검사되는 방식
- Python, JavaScript, Ruby 등
- 장점 : 코딩 속도가 빠르고 유연한 프로그래밍이 가능
- 단점 : 타입 관련 오류를 프로그램 실행 중에만 발견 가능하여 예기치 않은 오류가 발생하기 쉬움

타입스크립트의 데이터 타입 종류

- 타입스크립트의 데이터 타입과 추론
 - **타입스크립트**: 자바스크립트에 **정적 타이핑** 기능 제공
 - 정적 타이핑 : 타입 선언 후 해당 타입의 값만 할당/반환
 - 변수의 타입을 컴파일 타임(코드 번역 시점)에 미리 결정하고 검사하는 방식
 - C, C++, Java 등
 - 장점 : 대규모 협업시 안정성이 높고 가독성이 좋음
 - 단점: 코드를 작성할 때 타입을 선언해야 하므로 번거로울 수 있음
 - **타입 추론(Type Inference)** : TypeScript는 타입 추론을 하기 때문에 타입 선언을 생략 가능
 - `let age = 20;` 으로 선언 및 할당시 TypeScript는 age의 값을 20으로 선언했기 때문에 타입을 number로 인식

타입스크립트의 데이터 타입 종류

타입 스크립트 기본 데이터 타입

타입	내용
number	숫자형 데이터. 정수와 부동 소수점 값을 모두 포함한다.
string	문자열 데이터 타입이다.
boolean	참 또는 거짓을 나타내는 불리언 데이터 타입이다.
null	값이 없음을 명시하는 타입이다.
undefined	변수에 값이 할당되지 않았음을 나타내는 타입이다.
symbol	고유하고 변경할 수 없는 데이터를 나타낸다. 주로 객체의 고유한 속성 키를 만들기 위해 사용된다.
bigint	큰 정수(2^{53} 이상)를 표현하는 데이터 타입이다.

객체 타입

타입	내용
객체(Object)	속성과 값으로 구성된 데이터 구조이다.
배열(Array)	여러 값들을 순차적으로 저장하는 데이터 구조이다.
함수(Function)	특정 작업을 수행하는 코드 블록이다.
튜플(Tuple)	고정된 길이와 타입의 배열이다.
인터페이스(Interface)	객체 구조의 형식을 정의하는 타입이다.
클래스(Class)	객체를 생성하기 위한 템플릿이다.

특수 타입

타입	내용
유니온 타입(Union Type)	여러 타입 중 하나의 값을 가질 수 있는 타입이다.
열거형(Enum)	열거할 수 있는 값들을 정의하는 타입이다.
any	모든 타입을 허용하는 매우 유연한 타입이다.
unknown	타입을 알 수 없는 값에 대한 타입이다.
void	반환 값이 없음을 명시하는 타입이다.
never	절대 반환되지 않는 값에 대한 타입이다.

JavaScript와 TypeScript 데이터 타입 비교

TypeScript에는 JavaScript 기본 자료형 외에
코드를 더 안전하게 만들기 위한 다양한 타입 기능이 있다.

JavaScript와 TypeScript 데이터 타입 비교

구분	JavaScript	TypeScript	설명
문자열	string	string	문자 데이터
숫자	number	number	정수, 실수 모두 포함
논리값	boolean	boolean	true, false
값 없음	undefined	undefined	값이 할당되지 않음
빈 값	null	null	의도적으로 값이 없음을 표현
큰 정수	bigint	bigint	매우 큰 정수
고유값	symbol	symbol	고유한 식별자 생성
객체	object	object	객체, 배열, 함수 등의 기반
배열	Array	number[], Array<number> 등	TS에서는 요소 타입까지 지정
함수	function 객체	함수 타입 지정 가능	매개변수와 반환값 타입 지정
모든 타입 허용	없음	any	타입 검사를 사실상 하지 않음
알 수 없는 타입	없음	unknown	타입 확인 후 사용해야 함
반환값 없음	없음	void	함수 반환값이 없음을 표현
절대 발생하지 않음	없음	never	정상적으로 값이 만들어질 수 없음
고정된 배열 구조	없음	tuple	위치별 타입이 정해진 배열
열거형	없음	enum	여러 상수를 하나의 그룹으로 정의
여러 타입 중 하나	없음	Union	string number
여러 타입 결합	없음	Intersection &	여러 타입을 하나로 결합
특정 값 자체를 타입으로	제한적 개념	Literal Type	"admin", 200, true 등

JavaScript와 TypeScript의 자료형 비교

JavaScript vs TypeScript Data Types

JS

JavaScript — 동적 타입 (Dynamic Typing)

변수에 타입을 선언하지 않고, 실행 시 저장된 값에 따라 타입이 결정됨

Primitive Type (원시 타입)

string	문자열
number	숫자
boolean	참 / 거짓
undefined	값이 할당되지 않은 상태
null	의도적으로 값이 없음
bigint	큰 정수
symbol	고유 식별값

Object Type (객체 타입)

[]	Array	배열
f()	Function	함수
{}	Object	객체

```
JavaScript 예제 JS
let value = 10; // number
value = "Hello"; // string
value = true; // boolean
```

✓ 같은 변수에 서로 다른 타입의 값을 저장할 수 있음

JavaScript와 TypeScript의 관계

JavaScript
기본 자료형

+

TypeScript
정적 타입 시스템
(개발 단계의 타입 검사)



TypeScript
(JavaScript + Static Type System)

“TypeScript는 JavaScript의 자료형을 그대로 사용하면서 정적 타입 시스템을 추가한 언어이다.”

TS

TypeScript — 정적 타입 (Static Typing)

JavaScript 자료형을 그대로 사용하고, 개발 단계에서 타입 검사 기능을 제공

JavaScript의 기본 타입을 그대로 사용

string	number	boolean	undefined
null	bigint	symbol	object

TypeScript의 주요 타입 기능 (개발 단계의 타입 시스템)

Array 배열 타입	Tuple 고정 길이 배열	any 모든 타입 (타입 검사 제외)	unknown 알 수 없는 타입	void 값이 없는 타입	
never 절대 발생하지 않는 타입	Union Type 여러 타입 중 하나 (A B)	Literal Type 특정 값만 허용 ('A' 'B')	Interface 객체 구조 정의	Type Alias 타입에 별칭 부여	Generic 재사용 가능한 일반 타입

```
TypeScript 예제 TS
let value: number = 10;
value = 20; // OK
value = "Hello"; // Error (타입 오류)
```

✓ 타입 오류를 실행 전에 확인
개발 단계에서 타입을 검사하여 오류를 미리 발견할 수 있습니다.

구분	JavaScript	TypeScript
타입 방식	동적 타입 (Dynamic Typing)	정적 타입 (Static Typing)
타입 선언	일반적으로 하지 않음	: type으로 선언 가능 (예: let num: number)
타입 검사	실행 중 (런타임)	실행 전 (컴파일 타임)
타입 추론	실행 값 기준으로 타입이 결정됨	컴파일러가 타입을 추론
Interface / Type Alias / Generic	없음	지원 (개발 단계의 타입 시스템 기능)

- 💡 꼭 기억하세요!
- TypeScript에 JavaScript와 완전히 별개의 새로운 실행 자료형이 생기는 것은 아닙니다.
 - Interface, Type Alias, Generic 등은 JavaScript 실행 시 새로운 값으로 존재하지 않습니다.
 - 이들은 TypeScript가 개발 단계에서 코드의 타입과 구조를 검사하기 위해 제공하는 타입 시스템 기능입니다.

JavaScript의 기본 데이터 타입

- ECMAScript 기준으로 JavaScript의 언어 타입은 크게 **Primitive Type**과 **Object Type**으로 구분할 수 있음
- 원시 타입(Primitive Type) – string, number, boolean, undefined, null, symbol, bigint
- 객체 타입(Object Type) – 배열과 함수는 Object Type이며, function은 호출 가능한 Object 임

JavaScript – 원시타입(Primitive Type)

```
let name = "Kim";      // string
let age = 20;          // number
let student = true;    // boolean
let value;             // undefined
let data = null;       // null
let id = Symbol("id"); // symbol
let big = 12345678901234567890n; // bigint

let user = {
  name: "Kim",
  age: 20
}; // Object
```

JavaScript – 객체타입(Object Type)

```
let arr = [1, 2, 3] // 배열
console.log(typeof arr);

function hello(){ // 함수
  console.log("Hello");
}
console.log(typeof hello); // 객체 성격이지만 출력 결과는 function

let user = { // Object
  name: "Kim",
  age: 20
};
console.log(typeof user);
```

TypeScript에서 추가된 타입 – any

- any : 어떤 타입이든 허용, 사실상 타입 검사를 하지 않음
 - JavaScript 처럼 자유롭게 사용할 수 있지만 타입 검사의 장점이 없어짐
 - 가능하면 any 사용을 최소화하는 것이 좋음

TypeScript

```
let value:any = 10;
```

```
value = "hello";  
value = true;  
value = [1, 2, 3];
```

TypeScript에서 추가된 타입 – unknown

- unknown : 알 수 없는 타입, 타입 확인 후 사용해야 함
 - any와 비슷, 더 안전 하지만 바로 사용하면 안되며 타입을 확인하고 사용 해야함
 - 외부 API 데이터 처럼 타입이 확실하지 않을 때 unknown을 사용하여 안전하게 사용할 수 있음

TypeScript

```
let value: unknown;  
  
value = 10;  
value = "hello";  
  
if (typeof value === "string") {  
    console.log(value.toUpperCase());  
}
```

TypeScript에서 추가된 타입 – 함수 관련 타입 void, never

- void : 반환 값 없음, 함수 반환 값이 없음을 표현
 - 주로 함수의 반환 값 이 없을 때 사용
 - JavaScript에 void 연산자는 존재, TypeScript의 void 타입은 함수 반환 타입을 표현

TypeScript

```
function hello(): void {  
  console.log("Hello");  
}
```

- never : 절대 발생하지 않음, 정상적으로 값이 만들어질 수 없음
 - never는 정상적으로 값이 반환될 수 없는 경우를 표현

TypeScript

```
function error(message: string): never {  
  throw new Error(message);  
}
```

void 와 never 차이

void : 함수는 정상적으로 종료되지만 반환 값이 없음

never : 함수가 정상적인 반환 지점에 도달하지 않음

TypeScript에서 추가된 타입 – tuple

- tuple : 고정된 배열 구조, 위치별 타입이 정해진 배열
 - JavaScript에는 없는 TypeScript 대표적인 타입 기능

TypeScript

// 일반 배열

```
let userArray : (string | number)[] = ["Kim", 20];
```

// Tuple – 위치별 타입이 결정

// index 0 : string

// index 1 : number

```
let userTuple: [string, number] = ["Kim", 20];
```

```
let wrongUser: [string, number] = [20, "Kim"];    // 오류
```

TypeScript에서 추가된 타입 – enum

- enum : 열거형, 여러 상수를 하나의 그룹으로 정의
 - 관련된 여러 상수값을 묶을 수 있음
 - 문자열로 만들 수 있음

TypeScript – 하나의 그룹으로 정의

```
enum Direction {  
  Up,  
  Down,  
  Left,  
  Right  
}
```

```
let direction: Direction = Direction.Up;
```

TypeScript – 문자열 그룹

```
enum Role {  
  Admin = "ADMIN",  
  User = "USER",  
  Guest = "GUEST" ,  
}
```



TypeScript – 문자열 그룹을 Literal Type + Union Type

```
type Role = "ADMIN" | "USER" | "GUEST";
```

TypeScript에서 추가된 타입 – Union Type

- union : 여러 타입 중 하나로 TypeScript 중요한 기능
 - 함수에서도 많이 사용됨

TypeScript

```
let id: string | number; // string 또는 number
```

```
id = 100;  
id = "A100";
```

```
function printId(id: string | number) {  
  console.log(id);  
}
```

```
id = 100;  
printId(id);
```

```
id = "A100";  
printId(id);
```

TypeScript에서 추가된 타입 – Literal Type

- literal : 특정 값 자체를 타입으로 사용
 - 값 자체를 타입으로 지정할 수 있음

TypeScript

```
let direction: "left" | "right";
```

```
direction = "left";  
direction = "right";
```

```
direction = "up"; // 오류
```

```
type Status = "loading" | "success" | "error" ; // React와 Web API의 사용 예
```

TypeScript에서 추가된 타입 – Intersection Type

- intersection : 여러 타입을 하나로 결합

TypeScript

```
type Person = {  
  name: string;  
};  
  
type Employee = {  
  employeeId: number;  
};  
  
type Worker = Person & Employee; // Person + Employee -> Worker  
  
const user: Worker = {  
  name: "Kim",  
  employeeId: 1001  
};
```

interface 객체의 구조 정의

- interface는 객체가 어떤 속성을 가지고 있어야 하는지 구조를 정의
 - 선택적 속성 '?' : 반드시 존재하지 않아도 되는 속성
 - 읽기 전용 readonly : 값을 처음 설정한 후 변경하지 못하도록 함

TypeScript

```
interface User {  
  name: string;  
  age: number;  
}  
  
const user: User = {  
  name: "Kim",  
  age: 20  
};
```

TypeScript – 선택적 속성

```
interface User {  
  name: string;  
  age: number;  
  email?: string; // 선택적 속성  
}  
  
const user1: User = { // 사용가능  
  name: "Kim",  
  age: 20  
};  
  
const user2: User = { // 사용가능  
  name: "Lee",  
  age: 21,  
  email: "lee@test.com"  
};
```

TypeScript – 읽기 전용

```
interface User {  
  readonly id: number;  
  name: string;  
}  
  
const user: User = {  
  id: 1,  
  name: "Kim"  
};  
  
user.name = "Lee"; // 가능  
user.id = 2; // 오류
```

type 새로운 타입 이름을 정의

- type : 특정 타입에 새로운 이름을 붙여서 재사용

TypeScript

```
type UserName = string;

let name: UserName = "Kim";    // UserName 은 string 타입임
```

- 객체 구조도 정의 가능

TypeScript

```
type User = {
  name: string;
  age: number;
};

const user: User = {
  name: "Kim",
  age: 20
};
```

- type은 union 과 결합 : **type ID = string | number;**
- literal type 과 사용 : **type Status = "loading" | "success" | "error";**

interface와 type 비교

구분	interface	type
객체 구조 정의	가능	가능
함수 타입 정의	가능	가능
배열 타입 정의	가능	가능
Union Type	직접 사용 어려움	매우 적합
Literal Type	부적합	적합
Intersection	extends 중심	& 사용
확장	extends	&
동일 이름 재선언	병합 가능	불가능
주요 활용	객체, 클래스 구조	복합 타입, Union, Literal 등

권장 사용

객체 구조 정의 : **Interface**

여러 타입을 조합 : **Type**

TypeScript

```
interface User_interface {  
  name: string;  
  age: number;  
}
```

```
type User_type = {  
  name: string;  
  age: number;  
};
```

객체 타입 지정

- user 객체 타입 지정 예

JavaScript

```
const user = {  
  name: "Kim",  
  age: 20  
};
```

TypeScript

```
const user: {  
  name: string;  
  age: number;  
} = {  
  name: "Kim",  
  age: 20  
};
```

- 더 많이 사용되는 방법 - interface나 type을 많이 사용

TypeScript

```
interface User {  
  name: string;  
  age: number;  
}  
  
const user: User = {  
  name: "Kim",  
  age: 20  
};
```

TypeScript 추가 타입/기능	역할	예
any	모든 타입 허용	let x: any
unknown	알 수 없는 타입을 안전하게 처리	let x: unknown
void	함수 반환값 없음	(): void
never	정상적으로 반환되지 않음	(): never
Tuple	위치별 타입 지정	[string, number]
Enum	관련 상수 그룹	enum Role
Union	여러 타입 중 하나	string number
Intersection	여러 타입 결합	A & B
Literal Type	특정 값만 허용	"on" "off"
Interface / Type	객체 구조 정의	interface User

변수

변수

- 변수(variable): 값을 저장하고 저장한 값을 불러올 수 있게 하는 것
- 변수 선언(declaration): 변수를 만드는 행위
- **let으로 변수 선언하기**
 - 변수를 선언하는 방법
 - let(또는 const, var) 다음에 선언하려는 변수명(변수의 이름) 적음
 - 그 뒤에 대입(assignment, 할당) 연산자 = 입력, = 연산자 뒤에는 변수에 저장할 식 입력
 - let 변수명 = 식;
- 선언문 : let으로 시작하는 명령
- 초기화(initialization): 변수를 선언함과 동시에 값을 대입하는 행위

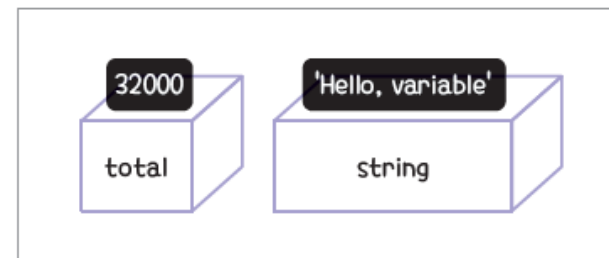
```
> let total = 5000 + 8000 + 10000 + 9000;  
< undefined
```

undefined는 JavaScript에서 "값이 정의되어 있지 않음"을 뜻함
변수 선언문의 반환 값이 없기 때문에 undefined로 표시

계산 실패한 것은 아님

```
> let string = 'Hello, variable';  
< undefined  
> string;  
< 'Hello, variable'
```

- **선언 Declaration**
let age;
- **초기화 Initialization**
let age = 20;
- **할당 Assignment**
age = 30;



메모리

• 변수명

- 변수의 값이 무엇인지 알려 주는 역할을 하므로 자세하게 짓기

• 변수명의 제약 사항

- 특수문자는 \$와 _만 사용할 수 있다.
- 숫자로 시작해서는 안 된다.
- 예약어(reserved word)는 변수명으로 사용할 수 없다.
- 위의 제약 사항을 어기지 않는다면 한글, 한자, 유니코드도 가능

구분	예약어	주요 의미
변수 선언	var, let, const	변수·상수 선언
조건문	if, else, switch, case, default	조건에 따른 분기
반복문	for, while, do	반복 처리
반복 제어	break, continue	반복문 종료·건너뛰기
함수	function, return	함수 선언 및 값 반환
클래스	class, extends, super	클래스와 상속
객체 관련	new, this, instanceof	객체 생성 및 참조
연산자	typeof, delete, in, void	타입·객체·속성 관련 연산
예외 처리	try, catch, finally, throw	오류 및 예외 처리
모듈	import, export	외부 모듈 가져오기·내보내기
비동기 처리	await, yield	비동기 함수·Generator 처리
디버깅	debugger	디버거 실행
기타	with	객체 스코프 관련 문법, 현재는 사용 권장 안 함
예약 리터럴	true, false, null	Boolean 및 null 값
향후/제한 예약어	enum, implements, interface, package, private, protected, public, static	향후 문법 또는 strict mode 등에서 제한

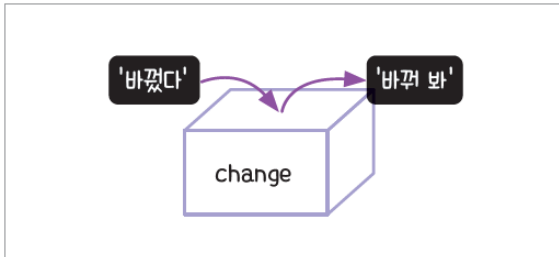
• 변수 값 수정

- 변수에 넣은 값을 바꿀 때: 대입 연산자를 사용해 새로운 값 입력

```
> let change = '바꿔 봐';  
< undefined
```

```
> change = '바꿨다';  
< '바꿨다'
```

```
> change;  
< '바꿨다'
```



메모리

- 변수에 넣은 값을 비울 때 : undefined를 대입하거나 null을 대입

• 변수 활용하기

- 변수는 계산된 값을 저장할 때
- 코드 중복을 줄일 때

- **const** 로 상수 선언하기

- const: 상수(constant)의 줄임말
- 한번 값을 대입하면 다른 값을 대입할 수 없다는 특성 때문에 **상수 선언 시 초기화(선언과 동시에 값을 대입하는 것) 하지 않으면 에러 발생**

```
> const value = '상수입니다.';
< undefined
> value = '바꿀 수 없습니다.';
Uncaught TypeError: Assignment to constant variable.
```

- const → 기본적으로 사용
- let → 값이 바뀌어야 할 때 사용
- var → 기존 코드 이해용, 새 코드에서는 가급적 사용하지 않음

- **var** 알아보기

- var: 변수(variable)의 줄임말
- 변수문(variable statement): var로 변수를 선언한 문장
- 기존에 선언한 변수를 다시 선언해도 에러가 발생하지 않음
- 예약어에 사용하는 단어를 변수명으로 사용하지 않음

var, let, const 변수 선언 방식의 차이

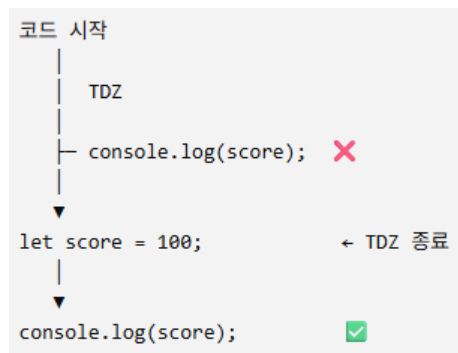
- 현대 JavaScript에서는 기본적으로 **const**를 사용, 값이 변경되어야 할 때 **let**을 사용
- 값이 변경된다면 **let**, 고정이라면 **const** 사용, var는 특별한 이유가 없다면 사용하지 않음

구분	var	let	const
등장 시기	초기 JavaScript	ES6(ES2015)	ES6(ES2015)
스코프(Scope)	함수 스코프	블록 스코프	블록 스코프
값 재할당	가능	가능	불가능
같은 범위에서 재선언	가능	불가능	불가능
선언 전 접근	Undefined	ReferenceError	ReferenceError
호이스팅	선언 + undefined 초기화	호이스팅 되지만 TDZ(Temporal Dead Zone) 적용	호이스팅 되지만 TDZ(Temporal Dead Zone) 적용
선언과 동시에 값 할당	선택	선택	필수
현대 JS 사용 권장	가급적 사용하지 않음	값 변경 시 사용	기본 사용 권장

- Temporal Dead Zone : 변수가 선언 정보는 등록되어 있지만, 선언문에 도달하기 전까지 접근할 수 없는 영역

- let, const는 이 개념이 중요

```
console.log(score); // ❌ ReferenceError  
  
let score = 100;  
  
console.log(score); // ✅ 100
```



```
// a가 존재하기는 하지만 접근 불가  
// ——— TDZ ———
```

```
console.log(a); // ReferenceError  
  
let a = 10;  
// TDZ 종료  
  
console.log(a); // 10
```

(참고) 호이스팅 변수

- 호이스팅(Hoisting)

- JavaScript가 코드를 실행하기 전에 변수·함수 등의 선언 정보를 먼저 등록하는 동작을 설명하는 개념

JavaScript – var 사용시

```
console.log(username);  
  
var username = "Kim";
```

name을 선언하기 전에 사용

javascript가 코드를 실행하기 전에 var name 이라는 변수 선언을 먼저 인식했기 때문에 에러 발생 안함

```
E:\frontend-workspace\FE_WS>node "e:\frontend-workspace\FE_WS\src\ch7-0\datatype\hoisting.js"  
undefined
```

JavaScript – let 사용시

```
console.log(username);  
  
let username = "Kim";
```

```
E:\frontend-workspace\FE_WS>node "e:\frontend-workspace\FE_WS\src\ch7-0\datatype\hoisting-let.js"  
e:\frontend-workspace\FE_WS\src\ch7-0\datatype\hoisting-let.js:1  
console.log(name);  
      ^
```

```
ReferenceError: Cannot access 'name' before initialization  
    at Object.<anonymous> (e:\frontend-workspace\FE_WS\src\ch7-0\datatype\hoisting-let.js:1:13)  
    at Module._compile (node:internal/modules/cjs/loader:1872:14)  
    at Object.<.> (node:internal/modules/cjs/loader:2003:10)  
    at Module.load (node:internal/modules/cjs/loader:1594:32)  
    at Module._load (node:internal/modules/cjs/loader:1396:12)  
    at wrapModuleLoad (node:internal/modules/cjs/loader:255:19)  
    at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:154:5)  
    at node:internal/main/run_main_module:33:47
```

Node.js v24.19.0

var은 선언 시 undefined로 초기화 되어 에러 발생 안함

let은 선언은 등록되지만, 초기화 되지 않아 에러 발생

let, const도 호이스팅은 가능함

(참고) 호이스팅 함수

- 호이스팅은 변수에만 있는 개념은 아님
- **함수 선언문(Function Declaration)** 은 함수 이름 뿐만 아니라 **함수 자체가 실행전에 등록 됨**

JavaScript – Function Declaration

```
hello();  
  
function hello() {  
  console.log("Hello");  
}
```

```
E:\frontend-workspace\FE_WS>node "e:\frontend-workspace\FE_WS\src\ch7-0\datatype\hoisting-function.js"  
Hello
```

- var 함수 표현식은 TypeError 발생
- let·const 함수 표현식은 ReferenceError 발생 실행시 hello가 TDZ(Temporal Dead Zone, 일시적 사각지대)에 있음

JavaScript – Function Declaration

```
hello();  
  
var hello = function () {  
  console.log("Hello");  
};
```

실행전 상태 : var hello -> undefined

실행시 hello() 는 undefined(); 로

```
E:\frontend-workspace\FE_WS>node "e:\frontend-workspace\FE_WS\src\ch7-0\datatype\hoisting-function2.js"  
e:\frontend-workspace\FE_WS\src\ch7-0\datatype\hoisting-function2.js:1  
hello();  
^  
TypeError: hello is not a function  
    at Object.<anonymous> (e:\frontend-workspace\FE_WS\src\ch7-0\datatype\hoisting-function2.js:1:1)  
    at Module._compile (node:internal/modules/cjs/loader:1872:14)  
    at Object..js (node:internal/modules/cjs/loader:2003:10)  
    at Module.load (node:internal/modules/cjs/loader:1594:32)  
    at Module._load (node:internal/modules/cjs/loader:1396:12)  
    at wrapModuleLoad (node:internal/modules/cjs/loader:255:19)  
    at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:154:5)  
    at node:internal/main/run_main_module:33:47
```

Node.js v24.19.0

(참고) JavaScript가 코드를 대략 두 단계로 처리

- JavaScript 코드 실행

- ① 실행 준비 단계

- 1) 변수 선언 확인
 - 2) 함수 선언 확인
 - 3) 사용할 식별자 등록

- ② 코드 실행 단계

- 1) 위에서 아래로 실행
 - 2) 값 할당
 - 3) 함수 호출

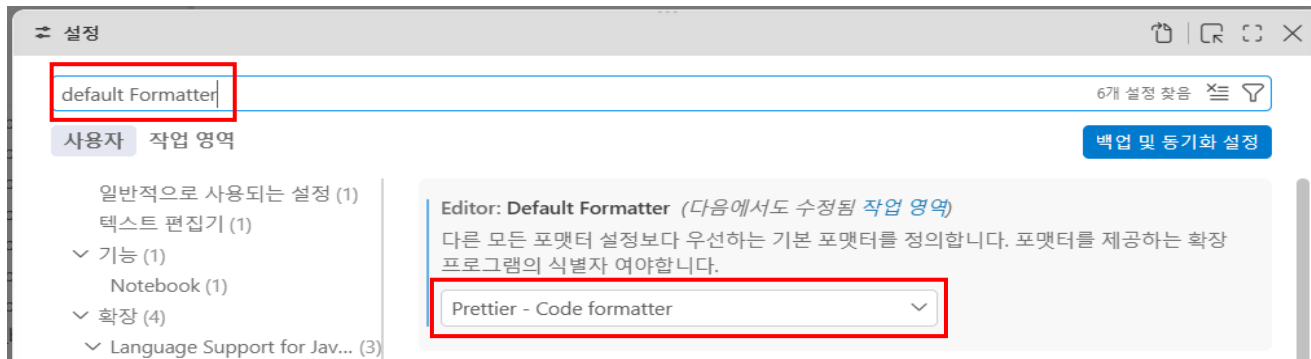
- **Hoisting 정리**

- JavaScript가 코드를 실행하기 전에 변수와 함수의 선언 정보를 먼저 등록하는 동작
 - var → 호이스팅 + undefined 초기화
 - let → 호이스팅 + TDZ
 - const → 호이스팅 + TDZ
 - function 선언문 → 함수 전체가 등록

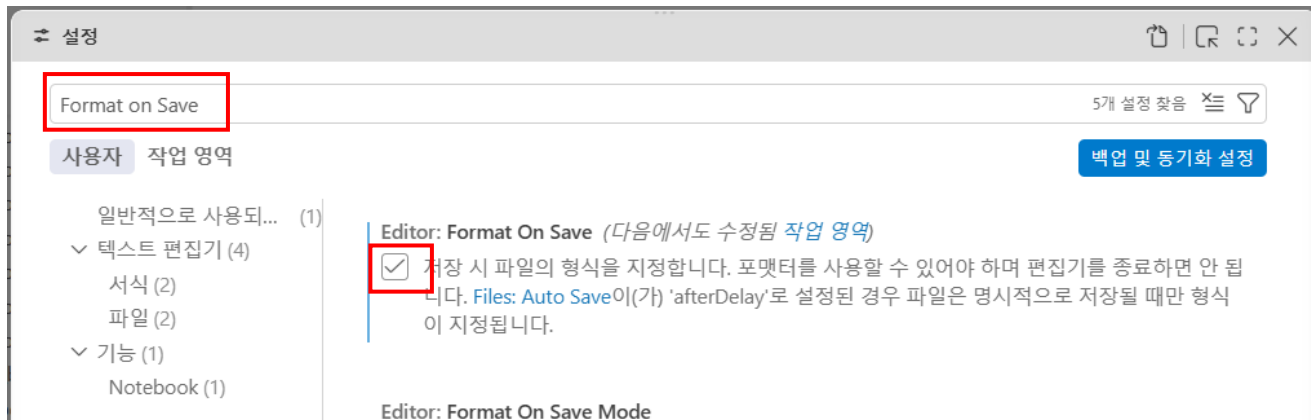
호이스팅을 이용해 선언 전에 변수를 사용하는 방식은 피한다!!

(참고) Prettier Extension 셋팅

1. 설정 열기 : Ctrl + ,
2. 검색창에 Default Formatter 입력
3. Editor : Default Formatter에서 Prettier – Code formatter 선택

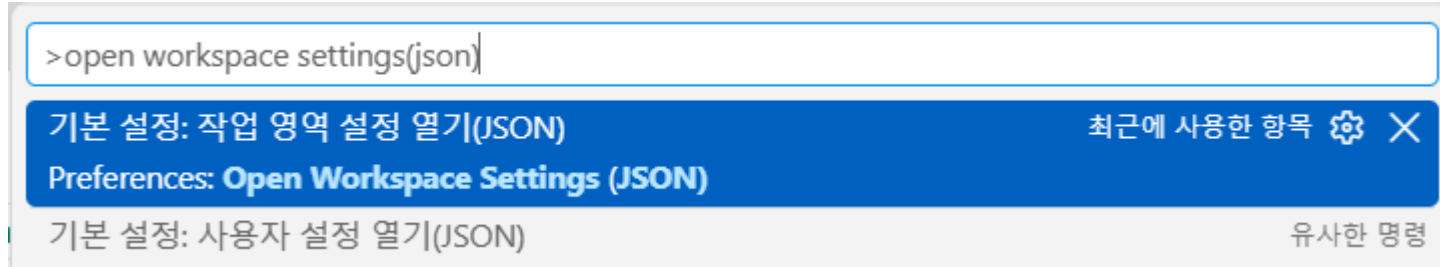


4. Format on Save 입력 후 체크박스 선택, 저장할 때 자동 정렬 설정 완료



(참고) Prettier Extension 셋팅

5. Ctrl + Shift + P 로 settings.json 설정



6. settings.json 파일에 prettier 설정 추가

settings.json > ...

```
{
  "code-runner.executorMap": {
    "javascript": "node",
    "typescript": "npx.cmd tsx"
  },
  "code-runner.runInTerminal": true,
  "code-runner.saveFileBeforeRun": true,
  "code-runner.ignoreSelection": true,
  "code-runner.clearPreviousOutput": true,
  "editor.fontSize": 16,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.formatOnSave": true
}
```

```
{
  "editor.fontSize": 16,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.formatOnSave": true
}
```